

Stage 3: Hybrid Index Construction (FAISS + BM25)

Project Presentation — Dense Semantic Search + Sparse Keyword Search

1. Executive Goal of Stage 3

In Stage 2, we converted approximately **1.04 GB of raw assets** into structured, metadata-enriched text passages saved in **data/processed/chunks.json**.

Scanning thousands of text chunks linearly for every query would be too slow and computationally inefficient. Stage 3 constructs two complementary local search indices:

- **Dense Vector Index (FAISS)**: semantic meaning and conceptual matching.
- **Sparse Keyword Index (BM25)**: exact lexical matching for course codes, names, dates, room numbers, and identifiers.

2. Technical Workflow Architecture

```
[ data/processed/chunks.json ]
████ [ SentenceTransformer: all-MiniLM-L6-v2 ]
█ █████ 384-D normalized embeddings
█ █████ [ FAISS IndexFlatIP ]
█ █████ faiss_index.bin
█
████ [ BM25 Tokenizer: lowercase lexical tokens ]
████ inverted frequency index
████ [ BM25Okapi ]
████ bm25_index.pkl
```

3. Detailed Component Breakdown

Component A: Dense Semantic Indexing (faiss_index.bin)

Embedding Model: `sentence-transformers/all-MiniLM-L6-v2`, a lightweight bi-encoder optimized for semantic similarity.

Vector Dimensionality: Every chunk is mapped into a continuous **384-dimensional** embedding space.

FAISS Indexing: Uses `faiss.IndexFlatIP` (Inner Product) with normalized vectors. For normalized vectors, inner product is equivalent to cosine similarity.

Key Advantage: Captures conceptual meaning. A query such as *“How much does college cost?”* can retrieve chunks discussing *“Academic Fee Structure”* or *“Hostel Dues”* even when the exact word “cost” is absent.

Component B: Sparse Keyword Indexing (bm25_index.pkl)

Algorithm: `BM25Okapi`, a probabilistic ranking method based on Term Frequency (TF), Inverse Document Frequency (IDF), and document-length normalization.

Tokenization: All chunk texts are tokenized after converting them to lowercase.

Key Advantage: Strong exact lexical matching where neural embeddings can be less reliable.

- **Course codes:** CSL202, ECL301
- **Faculty and administrative names**
- **Room identifiers:** BS-002, Lab 3
- **Exact dates and notice numbers**

4. Why Both Indices Are Required

FAISS is strong when users express an idea using different words from the source. BM25 is strong when the query contains a precise identifier that should match literally.

Example Query	FAISS Strength	BM25 Strength
“How much does college cost?”	Finds fee-related concepts and synonyms	May miss when exact wording differs
“What is CSL202?”	May understand course context	Strong exact match for CSL202

5. Summary Table for Defense & Presentation

Search Engine	Method / Model	Primary Strength	File Generated
Dense Vector Index	FAISS + all-MiniLM-L6-v2	Conceptual & semantic matching	data/processed/faiss_index.bin
Sparse Keyword Index	BM25 / BM25Okapi	Exact term & token matching	data/processed/bm25_index.pkl
Raw Embeddings Array	NumPy binary serialization	Persistent vector cache	data/processed/embeddings.npy

6. Output Deliverables

- **faiss_index.bin** — Dense vector store containing indexed 384-dimensional vectors.
- **bm25_index.pkl** — Serialized sparse keyword index for lexical retrieval.
- **embeddings.npy** — Persistent NumPy embedding array used as a vector cache for fast re-indexing.

Stage 3 outcome: The processed corpus is searchable through both semantic and exact-keyword retrieval. With both indices serialized and verified on disk, the pipeline is prepared for **Stage 4: Multi-Stage Hybrid Retrieval & Cross-Encoder Re-Ranking**.